



**BERLIN SCHOOL OF
BUSINESS & INNOVATION**

Essay Assignment Title

Stress Testing Attention and Uncertainty for Compact Polyp Segmentation

Programme title

MSc Data Analytics

Name

Rio Roy

Year

2026

Main report word count

2,736 words

Contents

Abstract

1 Introduction

2 Literature Review

3 Problem Formulation

4 Dataset Preparation

5 Model Architecture

6 Implementation

7 Model Evaluation and Discussion

8 Conclusion

References

Appendix

Academic Integrity Declaration

Submission must comply with the BSBI/UCA academic-integrity requirements. The required learner-declaration fields, including proofreading and AI-use disclosures, should be completed truthfully in the official submission form on Canvas. All academic sources used in this report are listed in the reference section.

Student name: Rio Roy Signature: _____ Date: _____

Abstract

This project tests two ideas that are often accepted too quickly in medical image segmentation: that attention always improves a model, and that an uncertainty map can reliably identify unsafe predictions. I used Kvasir-SEG, which contains 1,000 colonoscopy images with expert-checked masks. Compact U-Net and Attention U-Net models were trained with the same 800/100/100 split, settings and evaluation process across three random seeds. This produced 300 paired test observations for each architecture. For seed 42, U-Net achieved an aggregated Dice score of 0.6687, compared with 0.6634 for Attention U-Net. Across all three seeds, mean per-image Dice was 0.6253 for U-Net (95% bootstrap CI 0.5959-0.6550) and 0.6029 for Attention U-Net (0.5755-0.6310). The paired Wilcoxon test favoured U-Net ($p=0.0222$), showing that the attention model was less stable between runs. Attention added only 1.13% more parameters, but CPU inference was about 20.5% slower. I also tested Monte Carlo dropout using 12 predictions per image. Its uncertainty score had only a weak, non-significant relationship with segmentation error (Spearman $\rho=0.153$, $p=0.128$), and removing uncertain cases did not improve performance consistently. The main finding is therefore honest but useful: in this compact experiment, attention and basic MC-dropout uncertainty did not provide a reliable advantage. The system is not ready for clinical use and needs higher-resolution training, external validation and better uncertainty calibration.

1 Introduction

Medical image segmentation labels every pixel so that the exact shape of an anatomical structure or abnormal region can be measured. In this project, the target is the boundary of a polyp in a colonoscopy image. A reliable mask could support size measurement, treatment planning and consistent review. Manual outlining, however, takes time and can differ between clinicians. The images are also difficult because they may contain glare, weak contrast, irregular polyp shapes and large differences in scale. For these reasons, separating polyp tissue from the background is more challenging than it first appears.

Encoder-decoder networks are well suited to this task because they learn the wider image context and then rebuild a detailed mask. U-Net is a strong baseline because its skip connections pass fine spatial information from the encoder to the decoder (Ronneberger, Fischer and Brox, 2015). Attention gates may improve this process by reducing irrelevant features before they enter the decoder (Oktay et al., 2018). At the same time, attention adds computation and another

component that must be trained correctly. My aim was to compare U-Net and Attention U-Net under exactly the same compact protocol and then check whether the uncertainty output was genuinely related to model error. The central question was simple: does attention produce enough improvement to justify its extra complexity in this experiment?

2 Literature Review

Ronneberger, Fischer and Brox (2015) designed U-Net with a contracting path that learns context and an expanding path that restores location detail. The skip connections join features from both paths, which helps the model recover boundaries lost during pooling. This design works well when medical annotations are limited. Its main weakness is that the skip connections pass all selected features, including background information that may not help the final mask.

Oktay et al. (2018) introduced attention gates to make skip connections more selective. Their pancreas segmentation study showed that the gates could improve sensitivity with limited additional cost. This improvement cannot be assumed for every dataset because attention may also suppress small or unclear target regions. DeepLabV3 takes a different approach by using atrous convolution and atrous spatial pyramid pooling to capture objects at several scales (Chen et al., 2017). It is powerful, but a large backbone would require more computation than the compact models used here. nnU-Net automatically selects preprocessing, architecture and training settings from the dataset (Isensee et al., 2021). Its success shows that a carefully designed pipeline can matter as much as a new layer. However, using it in this experiment would change too many factors at once and make the attention comparison less clear.

Kvasir-SEG provides 1,000 polyp images and matching pixel masks for reproducible research (Jha et al., 2020). The original paper reported a test Dice score of 0.7878 and mean IoU of 0.7778 for ResUNet. These results provide useful context, but they are not a direct comparison with my models because the image resolution, split, training procedure and model size are different. Gal and Ghahramani (2016) showed that keeping dropout active during inference can approximate Bayesian sampling. Repeated predictions can then reveal where the model is unstable. This idea is attractive in medicine, but the uncertainty still has to match real errors before it can support human review.

Many model comparisons change the architecture, image size, augmentation and training settings at the same time. When performance improves, it becomes difficult to know which change caused it. Small medical datasets add another problem because the split and random initialisation can affect the result. I therefore repeated the same experiment with three seeds instead of reporting

only the best run. I also treated uncertainty as a result that had to be tested. If it is useful for referral, higher uncertainty should appear on worse predictions, and retained performance should improve when uncertain cases are removed. A convincing heatmap by itself is not enough.

3 Problem Formulation

Each RGB colonoscopy image, x , is converted into a binary mask, y . A value of one represents polyp tissue and zero represents background. The model returns a probability for each pixel, and I used a fixed threshold of 0.5 to create the final mask. Because most pixels are background, pixel accuracy can look high even when part of the polyp is missed. Dice and IoU were therefore used as the main overlap measures. Precision was included to show false-positive control, while recall showed how much true polyp tissue the model recovered.

The experiment followed three main rules. First, all results had to come from a real medical dataset rather than synthetic images. Second, both models had to use the same split, preprocessing, loss function, optimiser, number of epochs and threshold. This kept the architecture as the main difference. Third, an uncertain output could not be treated as a diagnosis. This is a research prototype, not a medical device, and any clinical use would require independent validation, governance and direct oversight from qualified specialists.

I tested two hypotheses. H1 expected Attention U-Net to improve paired per-image Dice compared with U-Net. H2 expected MC-dropout variance to identify failure risk: uncertainty should increase with Dice error, and performance should improve as the most uncertain cases are referred for review. Both hypotheses were fixed before interpreting the final results. A result that did not support them was still kept and discussed instead of changing the definition of success afterwards.

4 Dataset Preparation

The dataset was obtained from the official Simula Kvasir-SEG page: <https://datasets.simula.no/kvasir-seg/>. It contains 1,000 JPEG colonoscopy images and 1,000 masks with matching filenames. Image sizes range from 332 x 487 to 1,920 x 1,072 pixels. An engineer and a medical doctor outlined the polyp regions, and an experienced gastroenterologist reviewed the annotations (Jha et al., 2020). The dataset is available for research and education and requires citation of the original paper.

Stage	Images	Purpose
Training	800	Parameter optimisation and

		horizontal-flip augmentation
Validation	100	Best-epoch selection
Test	100	Final held-out evaluation

I sorted the filenames, shuffled them with seed 42 and created separate training, validation and test sets with no overlap. Images were resized to 96 x 96 with bilinear interpolation and scaled to the range [0,1]. Masks were resized with nearest-neighbour interpolation so that resizing did not create new label values, and they were then thresholded at 127. Random horizontal flipping was applied only to the training pairs. Before training, checks confirmed that all 1,000 images had a matching mask and that no file appeared in more than one split.

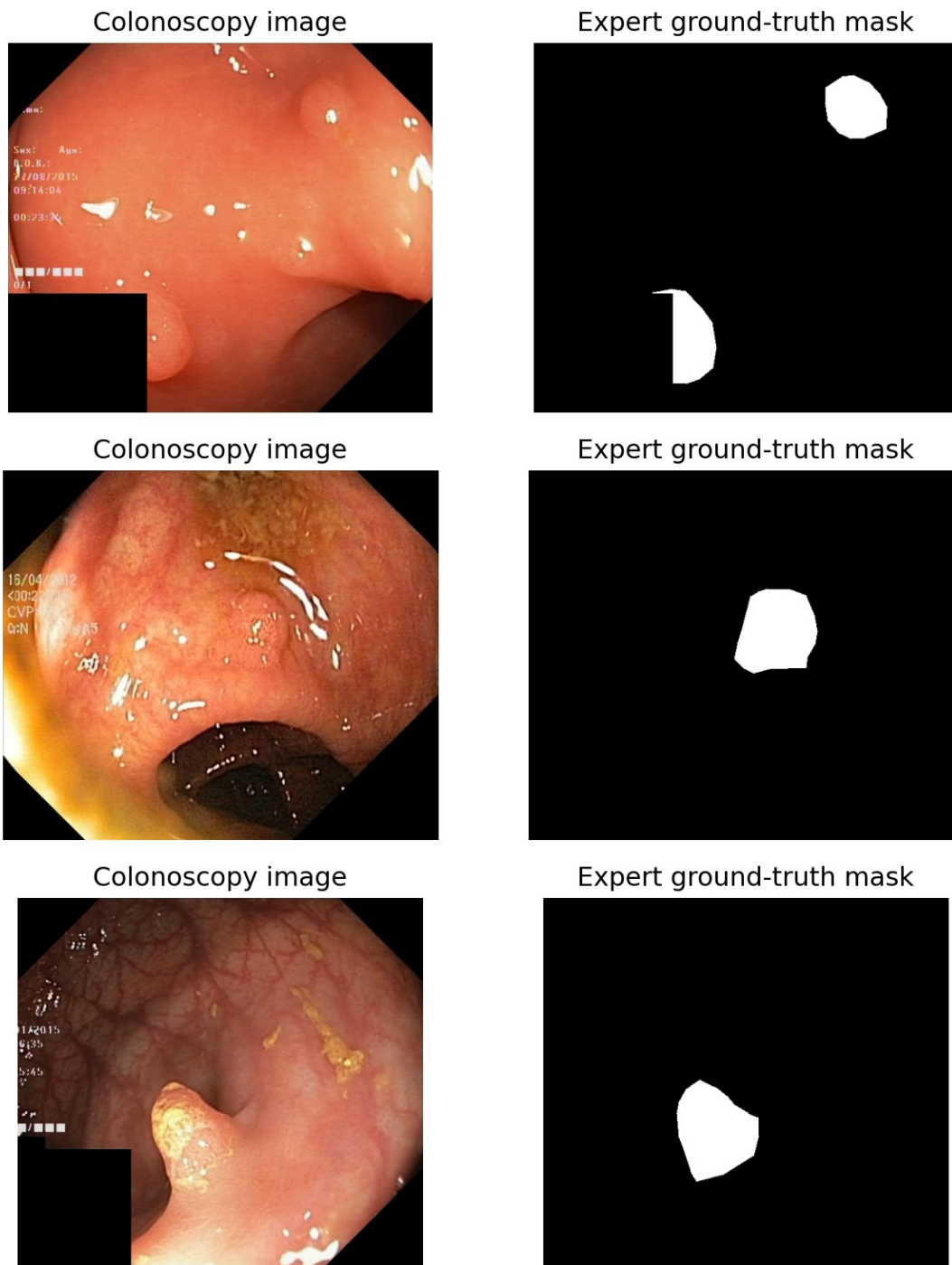


Figure 1. Examples from Kvasir-SEG showing each image and its paired expert mask.

5 Model Architecture

The baseline was a compact three-level U-Net. Its channel widths were 8, 16 and 32, followed by a 64-channel bridge. Each block contained two 3 x 3 convolutions with batch normalisation and

ReLU. Max pooling reduced the image dimensions, while transposed convolutions increased them again in the decoder. Encoder features were joined to the decoder through skip connections, and a final 1×1 convolution produced one logit for each pixel. The model had 121,177 trainable parameters.

The second model kept the same encoder-decoder structure but added a learned attention gate to each skip connection. A 1×1 projection aligned the decoder and encoder features. ReLU and sigmoid then produced a spatial weight that controlled how much of the skip feature was passed forward. Dropout of 0.20 was added to the bridge and decoder blocks so that uncertainty could be estimated later. Attention U-Net had 122,552 parameters, which was only 1,375 more than U-Net, or 1.13%.

U-Net is simple, fast and effective at recovering spatial detail, but its skip connections may carry background features into the decoder. Attention U-Net can reduce some of this noise, although it also adds computation and can become more sensitive to training conditions. DeepLabV3 would provide stronger multi-scale context, and nnU-Net would automate many design choices, but either option would change much more than the attention mechanism. I therefore kept this experiment focused on a controlled comparison rather than trying to reproduce every advanced architecture.

```

1 class AttentionGate(nn.Module):
2     def __init__(self, gate_channels: int, skip_channels: int, inter_channels: int):
3         super().__init__()
4         self.gate = nn.Conv2d(gate_channels, inter_channels, 1, bias=False)
5         self.skip = nn.Conv2d(skip_channels, inter_channels, 1, bias=False)
6         self.score = nn.Sequential(nn.ReLU(inplace=True), nn.Conv2d(inter_channels, 1, 1), nn.Sigmoid())
7
8     def forward(self, gate, skip):
9         attention = self.score(self.gate(gate) + self.skip(skip))
10        return skip * attention
11
12
13 class UNet(nn.Module):
14     def __init__(self, base: int = 8, attention: bool = False, dropout: float = 0.0):
15         super().__init__()
16         self.attention = attention
17         self.enc1 = ConvBlock(3, base)
18         self.enc2 = ConvBlock(base, base * 2)
19         self.enc3 = ConvBlock(base * 2, base * 4)
20         self.pool = nn.MaxPool2d(2)
21         self.bridge = ConvBlock(base * 4, base * 8, dropout=dropout)
22         self.up3 = nn.ConvTranspose2d(base * 8, base * 4, 2, stride=2)
23         self.up2 = nn.ConvTranspose2d(base * 4, base * 2, 2, stride=2)
24         self.up1 = nn.ConvTranspose2d(base * 2, base, 2, stride=2)
25         if attention:
26             self.att3 = AttentionGate(base * 4, base * 4, base * 2)
27             self.att2 = AttentionGate(base * 2, base * 2, base)
28             self.att1 = AttentionGate(base, base, max(base // 2, 1))
29         self.dec3 = ConvBlock(base * 8, base * 4, dropout=dropout)
30         self.dec2 = ConvBlock(base * 4, base * 2, dropout=dropout)
31         self.dec1 = ConvBlock(base * 2, base, dropout=dropout)
32         self.out = nn.Conv2d(base, 1, 1)
33
34     def forward(self, x):
35         e1 = self.enc1(x)
36         e2 = self.enc2(self.pool(e1))
37         e3 = self.enc3(self.pool(e2))
38         bridge = self.bridge(self.pool(e3))
39         d3 = self.up3(bridge)
40         s3 = self.att3(d3, e3) if self.attention else e3
41         d3 = self.dec3(torch.cat([d3, s3], dim=1))
42         d2 = self.up2(d3)
43         s2 = self.att2(d2, e2) if self.attention else e2
44         d2 = self.dec2(torch.cat([d2, s2], dim=1))
45         d1 = self.up1(d2)
46         s1 = self.att1(d1, e1) if self.attention else e1
47         d1 = self.dec1(torch.cat([d1, s1], dim=1))
48         return self.out(d1)

```

Figure 2. Screenshot of the implemented attention-gated U-Net architecture.

Model	Parameters	Relative size
U-Net	121,177	Baseline
Attention U-Net	122,552	+1.13%

6 Implementation

I implemented the experiment in Python 3 using PyTorch 2.14. Both models were trained with Adam, a learning rate of 0.001, batch size 16 and ten epochs. The loss combined binary cross-entropy and soft Dice loss. Binary cross-entropy gave stable feedback for each pixel, while Dice loss focused training on foreground overlap. At the end of every epoch, I calculated validation loss and the pixel-level confusion counts. The checkpoint with the highest validation Dice was restored before the final test evaluation.

Metrics were calculated from the total true-positive, false-positive, false-negative and true-negative pixels across the test set. This prevented images with very small and very large masks from being treated as if their pixel counts were identical. Dice was calculated as $2TP/(2TP+FP+FN)$, and IoU as $TP/(TP+FP+FN)$. I also reported precision, recall, specificity and pixel accuracy so that different error types remained visible. Parameter counts came directly from the trainable tensors. CPU inference time was averaged over ten warmed-up batches and divided by the number of images.

To check whether the conclusion depended on one lucky run, I repeated the full protocol with seeds 7, 42 and 123. Per-image Dice and IoU produced 300 paired observations for each model. I used 5,000 bootstrap samples to estimate 95% confidence intervals for the mean. A paired Wilcoxon signed-rank test then compared the Attention U-Net and U-Net Dice values. These results apply to this protocol only. Because the images come from related acquisition conditions, the confidence intervals may still underestimate uncertainty at the dataset level.

```

1 class KvasirDataset(Dataset):
2     def __init__(self, names: list[str], augment: bool = False):
3         self.names = names
4         self.augment = augment
5
6     def __len__(self) -> int:
7         return len(self.names)
8
9     def __getitem__(self, index: int):
10        name = self.names[index]
11        image = Image.open(DATA_DIR / "images" / name).convert("RGB")
12        mask = Image.open(DATA_DIR / "masks" / name).convert("L")
13        image = image.resize((IMAGE_SIZE, IMAGE_SIZE), Image.Resampling.BILINEAR)
14        mask = mask.resize((IMAGE_SIZE, IMAGE_SIZE), Image.Resampling.NEAREST)
15        image_array = np.asarray(image, dtype=np.float32) / 255.0
16        mask_array = (np.asarray(mask, dtype=np.float32) > 127).astype(np.float32)
17        if self.augment and random.random() < 0.5:
18            image_array = np.ascontiguousarray(image_array[:, ::-1])
19            mask_array = np.ascontiguousarray(mask_array[:, ::-1])
20        image_tensor = torch.from_numpy(image_array).permute(2, 0, 1)
21        mask_tensor = torch.from_numpy(mask_array)[None]
22        return image_tensor, mask_tensor, name

```

Figure 3. Screenshot of the paired image-mask loading and preprocessing code.

For uncertainty estimation, dropout remained active while the selected Attention U-Net produced 12 predictions for each image. Their mean probability created the final mask, while their pixel-level variance created the uncertainty map. This measures instability within the trained model, but it does not represent every source of clinical uncertainty. The notebook and training scripts are included so that the complete experiment, calculations and figures can be checked and reproduced.

7 Model Evaluation and Discussion

Model	Dice	IoU	Precision	Recall	Specificity	Pixel acc.
U-Net	0.6687	0.5022	0.5779	0.7932	0.9007	0.8849
Attention U-Net	0.6634	0.4964	0.6318	0.6984	0.9302	0.8963

The corresponding pixel-level confusion counts for the seed-42 test set are shown below, making the false-positive and false-negative trade-off explicit.

Model	True positive	False positive	False negative	True negative
U-Net	106,993	78,139	27,898	708,570
Attention U-Net	94,213	54,914	40,678	731,795

For seed 42, U-Net produced slightly better overlap: Dice was higher by 0.0052 and IoU by 0.0059. Attention U-Net increased precision by 0.0538 and specificity by 0.0295, but recall fell by 0.0947. This means that the attention model was more conservative. It marked fewer background pixels as polyp tissue, but it also missed more true polyp pixels. In screening, this trade-off matters more than the small increase in pixel accuracy from 0.8849 to 0.8963, because missing lesion tissue may be more serious than drawing a mask slightly too widely.

Model	Mean per-image Dice	95% bootstrap CI	Mean per-image IoU	95% bootstrap CI
U-Net	0.6253	0.5959-0.6550	0.5013	0.4721-0.5305
Attention U-Net	0.6029	0.5755-0.6310	0.4728	0.4457-0.5007

The three-seed experiment gave a clearer result than seed 42 alone. Mean per-image Dice for U-Net ranged from 0.6008 to 0.6525. Attention U-Net ranged from 0.5430 to 0.6408, showing greater instability. Across the 300 paired observations, the median Attention U-Net minus U-Net difference was -0.0060, and the Wilcoxon test gave $p=0.0222$. The bootstrap intervals overlap, so the practical size of the difference should not be exaggerated. However, the repeated results do show that attention was not consistently helpful within this training setup. If only seed 42 had been reported, this weakness would have been missed.

A statistically significant result is not automatically an important one. The median difference was small, the confidence intervals were wide, and the same images were evaluated under several separately trained models. The safest conclusion is limited to this experiment: there is evidence against a consistent attention advantage in a compact ten-epoch setup. A stronger follow-up should use repeated cross-validation or hierarchical analysis to separate variation caused by the images, data split and random initialisation. It should also examine small, flat and low-contrast polyps separately, because overall averages can hide clinically important failures.

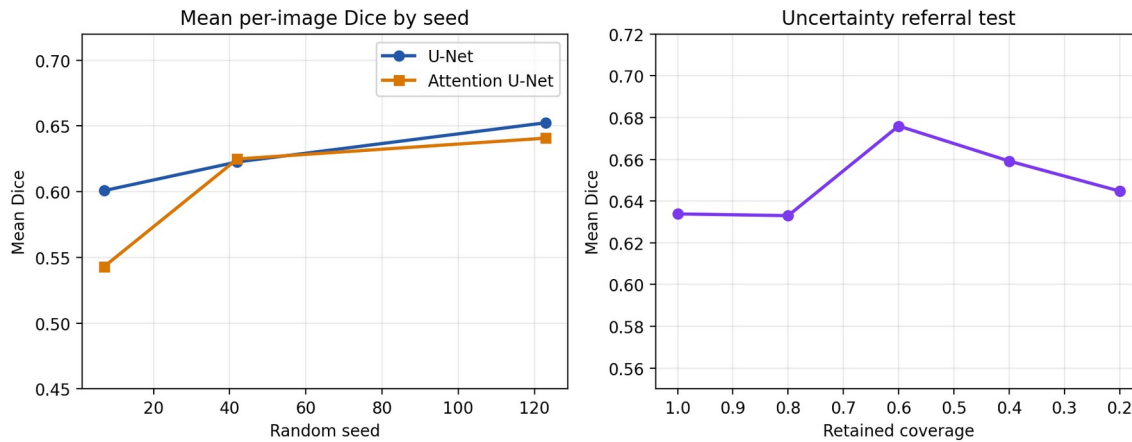


Figure 4. Repeated-seed Dice stability and uncertainty-based selective referral behaviour.

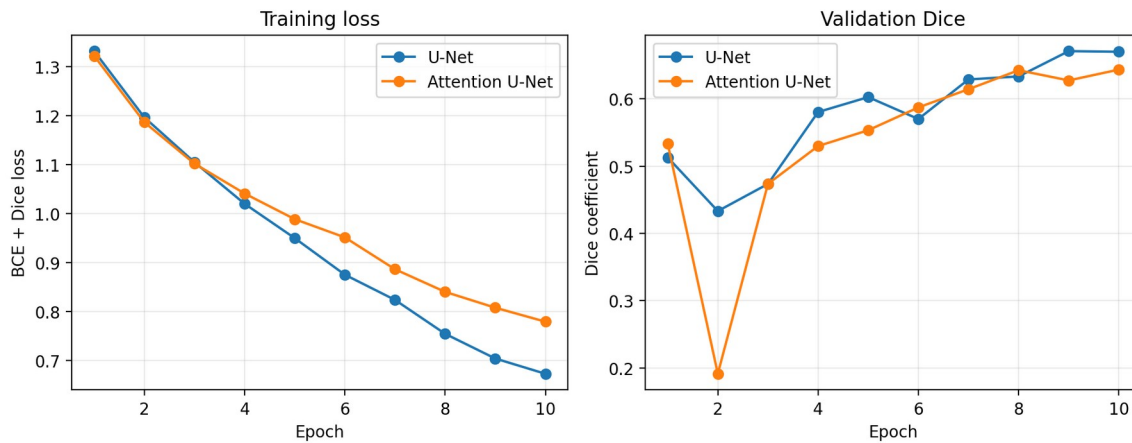


Figure 5. Training loss and validation Dice across ten epochs for seed 42.

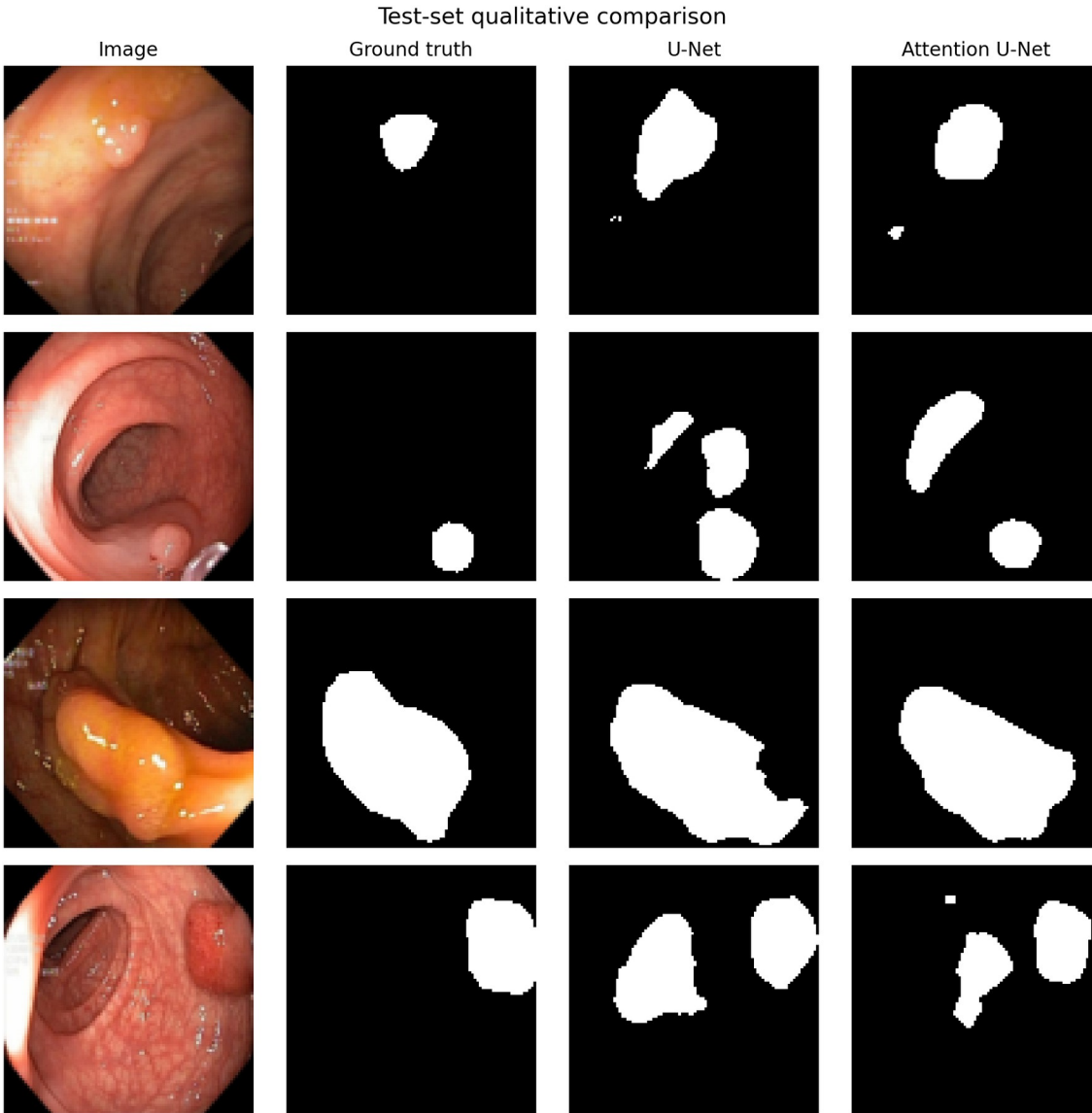


Figure 6. Qualitative comparison on four previously unseen test images.

The original Kvasir-SEG paper reported Dice 0.7878 and IoU 0.7778 for ResUNet (Jha et al., 2020), which is higher than both compact models in this project. This does not make the results directly comparable because the published experiment used different settings. My 96 x 96 images removed some fine boundary detail, ten epochs provided a short training period, and the networks were deliberately small. The gap points to clear next steps: higher-resolution input, residual blocks, longer scheduled training and stronger repeated validation.

Model	Training time s	Inference ms image	Interpretation
U-Net	78.65	1.461	Fastest and best

			overlap
Attention U-Net	88.02	1.761	Higher precision, slower inference

Attention increased the number of parameters by only 1.13%, but CPU inference rose from 1.461 to 1.761 ms per image, an increase of about 20.5%. This shows that parameter count alone does not describe scalability. Gating operations, memory access and the hardware implementation also affect speed. Both models were still lightweight, but these timings came from one CPU environment and should not be treated as expected performance on hospital systems or full-resolution colonoscopy video.

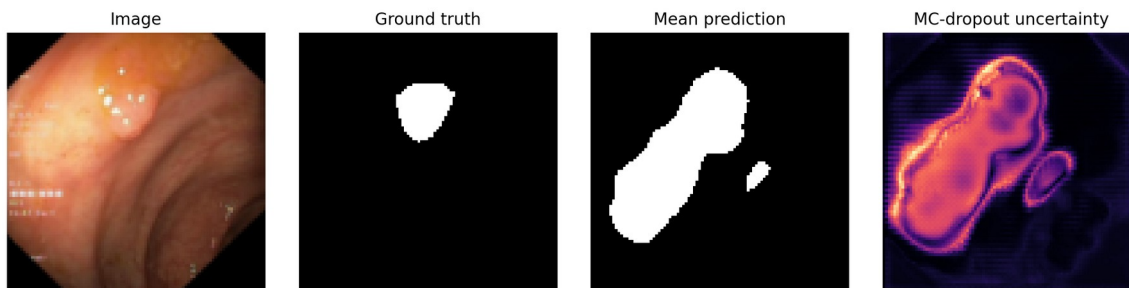


Figure 7. Attention U-Net mean prediction and variance from 12 Monte Carlo dropout passes.

The example uncertainty map had a mean variance of 0.0047 and a maximum of 0.0213. Across all 100 seed-42 test images, mean variance had only a weak relationship with Dice error (Spearman $\rho=0.153$, $p=0.128$). The referral test was also inconsistent. Mean Dice was 0.6338 with all cases, 0.6330 after removing the 20% most uncertain cases, 0.6759 at 60% coverage, 0.6591 at 40% coverage and 0.6448 at 20% coverage. If uncertainty ranked failures correctly, performance should have improved steadily as coverage decreased. It did not. The heatmap shows where repeated predictions disagree, but this score is not reliable enough to act as a clinical safety gate.

This result gives a useful direction for the next study. Taking the average variance across every pixel may hide uncertainty around the boundary and give too much weight to confident background pixels. Better scores could focus on the predicted contour, use predictive entropy inside the suspected lesion, calculate mutual information, or measure the share of pixels above a calibrated uncertainty level. These alternatives should be compared through a risk-coverage curve and tested again on an external dataset. That would develop this work into a focused study of safe referral rather than simply adding another model.

The study still has important limits. It used one dataset, no normal images without polyps, a low resolution of 96 x 96, three random seeds and no external domain test. It also did not include a boundary measure such as Hausdorff distance. Real deployment would require subgroup testing, privacy controls, audit logs and a validated rule for escalating uncertain cases. A model trained only on images containing polyps could create false alarms or false reassurance when applied to complete colonoscopy video. Final responsibility must remain with a gastroenterologist.

8 Conclusion

This project compared two compact segmentation models using a real, expert-annotated medical dataset and then checked whether the conclusion remained stable across repeated runs. U-Net achieved the stronger overall overlap with lower computational cost. Attention U-Net sometimes improved precision, but its Dice score varied more between seeds. The uncertainty experiment was equally important: MC-dropout variance did not show a significant relationship with segmentation error and did not improve retained performance consistently. The result is not that attention or uncertainty estimation never works. It is that neither should be trusted only because the architecture is more advanced or the heatmap looks convincing.

The next stage should train at 256 x 256 resolution or higher, use repeated cross-validation and test residual or pretrained encoders. External evaluation on a dataset such as CVC-ClinicDB is necessary to check whether the findings transfer to different images. Boundary Dice and Hausdorff distance should be added, while uncertainty should be compared using predictive entropy, mutual information, deep ensembles and boundary-focused scores. A serious publication should test one referral rule across datasets instead of adjusting it after seeing each result. Clinical validation, privacy protection, subgroup analysis and specialist control would still be required before deployment.

References

- Chen, L.-C., Papandreou, G., Schroff, F. and Adam, H. (2017) 'Rethinking atrous convolution for semantic image segmentation'. arXiv:1706.05587. Available at: <https://arxiv.org/abs/1706.05587> (Accessed: 16 September 2026).
- Gal, Y. and Ghahramani, Z. (2016) 'Dropout as a Bayesian approximation: Representing model uncertainty in deep learning', Proceedings of the 33rd International Conference on Machine Learning, pp. 1050-1059.

- Isensee, F., Jaeger, P.F., Kohl, S.A.A., Petersen, J. and Maier-Hein, K.H. (2021) 'nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation', *Nature Methods*, 18, pp. 203-211. doi: 10.1038/s41592-020-01008-z.
- Jha, D., Smedsrud, P.H., Riegler, M.A., Halvorsen, P., de Lange, T., Johansen, D. and Johansen, H.D. (2020) 'Kvasir-SEG: A segmented polyp dataset', *MultiMedia Modeling*, pp. 451-462. doi: 10.1007/978-3-030-37734-2_37.
- Okta, O. et al. (2018) 'Attention U-Net: Learning where to look for the pancreas'. arXiv:1804.03999. Available at: <https://arxiv.org/abs/1804.03999> (Accessed: 16 September 2026).
- Ronneberger, O., Fischer, P. and Brox, T. (2015) 'U-Net: Convolutional networks for biomedical image segmentation', *Medical Image Computing and Computer-Assisted Intervention*, pp. 234-241. doi: 10.1007/978-3-319-24574-4_28.
- SimulaMet (2026) Kvasir-SEG: Segmented polyp dataset for computer-aided gastrointestinal disease detection. Available at: <https://datasets.simula.no/kvasir-seg/> (Accessed: 16 September 2026).

Appendix

The supporting package contains `Rio_Roy_CVAI_Kvasir_Experiment.ipynb`, `train_models.py` and `publication_analysis.py`. It also contains the saved seed-42 metrics, training history and split record used in the report, together with a derived confusion-count record. The repeated-seed and uncertainty analyses are reproduced by running `publication_analysis.py` after `train_models.py` has generated the seed-42 checkpoints.

Reproducibility item	Recorded value
Random seeds	7, 42 and 123
Split per seed	800 train, 100 validation, 100 test
Input size	96 x 96 RGB
Batch size	16
Epochs	10
Optimiser	Adam, learning rate 0.001

Loss	Binary cross-entropy plus soft Dice
Threshold	0.5
Uncertainty	12 stochastic passes

To reproduce the work, download Kvasir-SEG from the source cited in Section 4 and place its images and masks in matching folders. Run `train_models.py` first; it generates the seed-42 checkpoints, metrics, split record and figures. Then run `publication_analysis.py` to reproduce the repeated-seed, bootstrap, Wilcoxon and risk-coverage analyses. The notebook can be used to inspect the saved outputs. Training time is processor-specific and should not be treated as a hardware-independent benchmark.